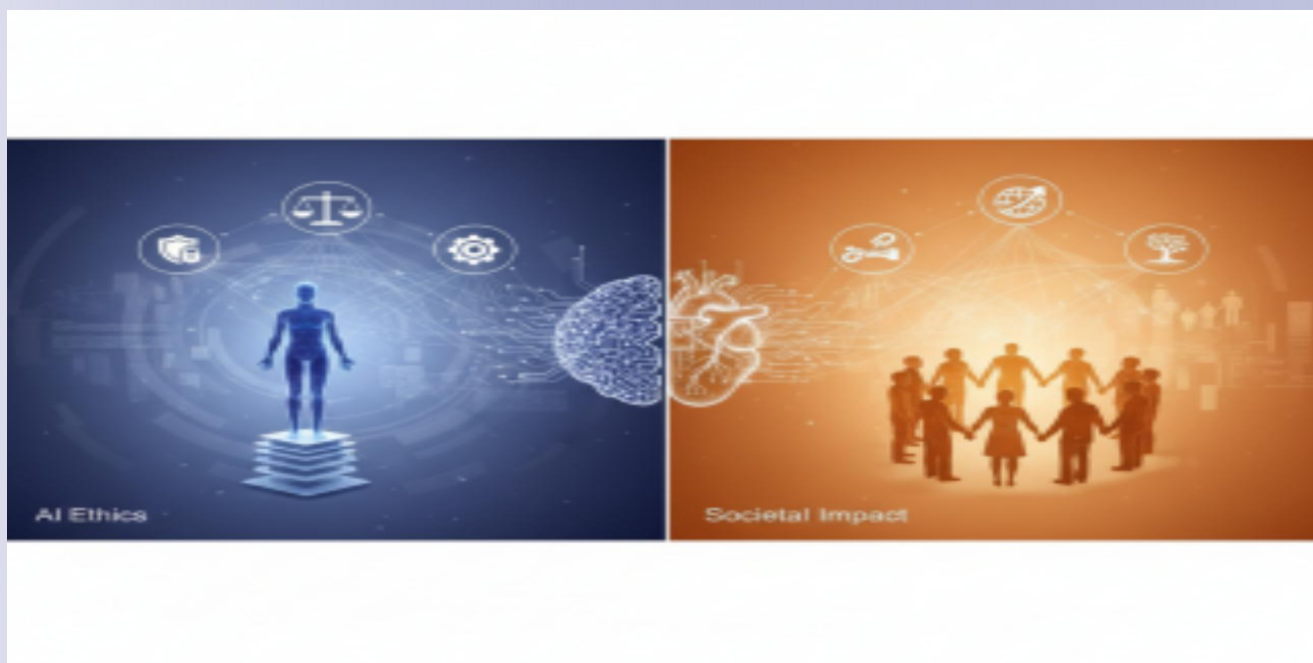
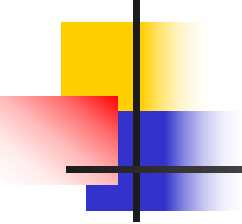


第3章 神经网络基础



湖北大学 计算机学院
王雷春



目 录

3.1 从生物神经元到人工神经元

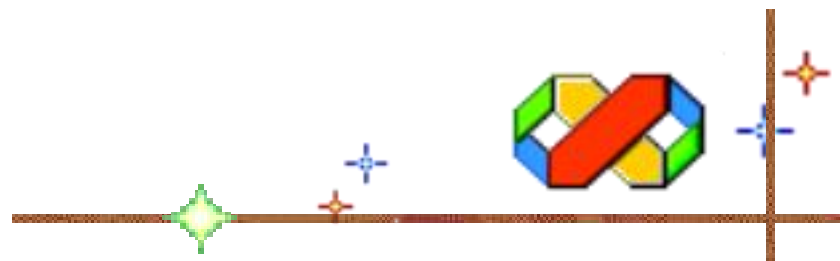
3.2 单层感知机和多层感知机

3.3 模型评估和优化

3.4 拟合问题和正则化

3.5 深度学习网络

3.6 PyTorch简介

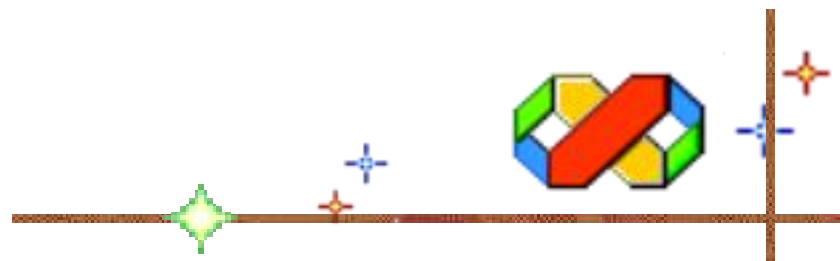


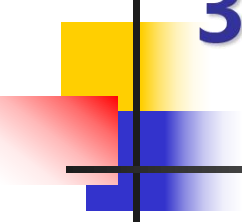
3.1 生物神经元到人工神经元

3.1.1 生物神经元

3.1.2 人工神经元

3.1.3 激活函数

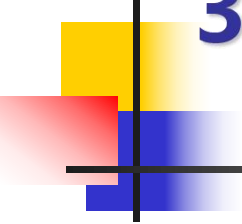




3.1.1 生物神经元

1. 含义

- ◆ 生物神经元（又称神经细胞）是神经系统的基本结构和功能单位，具有感受刺激、产生兴奋并传导神经冲动的能力。
- ◆ 典型神经元由细胞体、树突和轴突三部分组成，通过突触与其他神经元或效应器细胞形成连接，实现信息传递与处理。
- ◆ 人类大脑中约有860亿个神经元，它们通过复杂的连接形成一个高度并行、低功耗的信息处理网络。
- ◆ 理解生物神经元的工作原理，是理解人工神经网络设计思想的起点。



3.1.1 生物神经元

2. 特点

生物神经元的主要特点如下：

- ◆ (1)高度并行：大脑中数以亿计的神经元同时处理信息。
- ◆ (2)可塑性：突触连接强度可随学习和经验发生变化，这是记忆和学习的神经基础。
- ◆ (3)低功耗：人脑整体功耗约20瓦，远低于同等计算能力的数字计算机。
- ◆ (4)容错性：局部神经元损伤不会导致整体功能崩溃，系统具有高度鲁棒性。

3.1.1 生物神经元

3. 结构

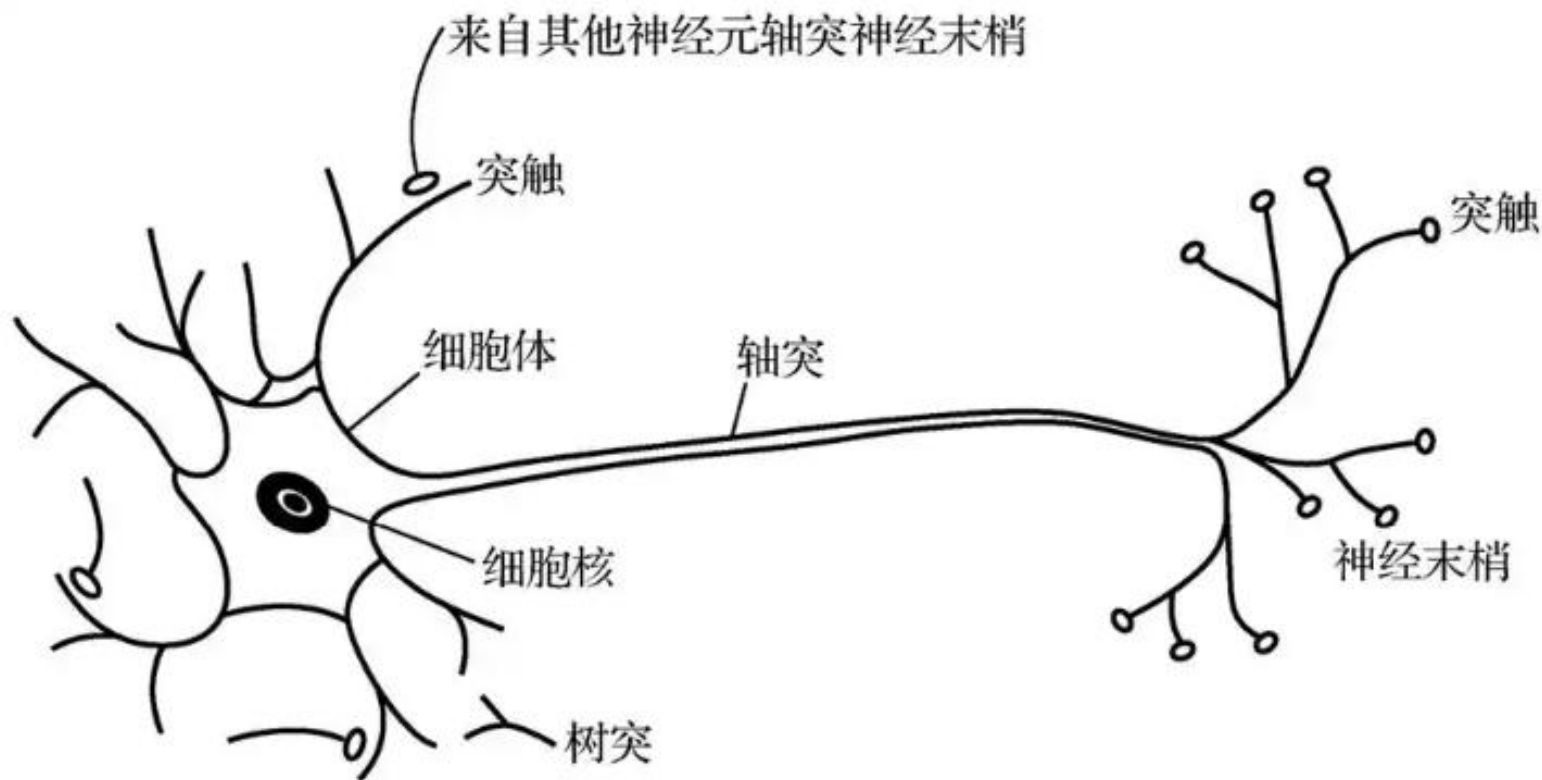
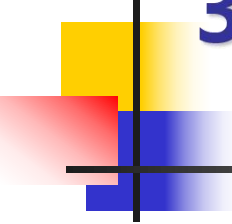


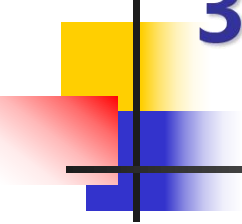
图 生物神经元



3.1.1 生物神经元

3. 结构

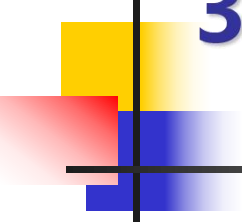
- ◆ 细胞体 (Soma)：神经元代谢和整合信息的中心，是神经元的“处理器”。
- ◆ 树突 (Dendrites)：从细胞体发出的树枝状突起，用于接收来自其他神经元的化学信号，并将其转化为电信号，是神经元的“输入端”。
- ◆ 轴突 (Axon)：从细胞体发出的细长突起，用于将胞体产生的动作电位（电脉冲）远距离传递给其他神经元或效应器（如肌肉），是神经元的“输出端”。
- ◆ 突触 (Synapse)：神经元与神经元之间，或神经元与效应器之间的功能性连接结构，通过释放神经递质实现化学信号传递，是神经元的输入输出接口。



3.1.1 生物神经元

4. 工作机制

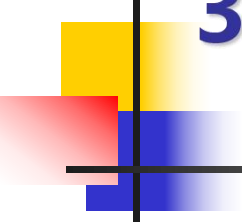
- ◆ (1)接收信号。树突接收来自其他神经元的化学信号（神经递质），将其转化为微弱的电信号。
- ◆ (2)整合信号。细胞体将所有兴奋性和抑制性信号进行总和。如果总和后的电位超过阈值（约-55mV），神经元被激活；否则无反应。
- ◆ (3)产生脉冲。轴突起始段一旦激活，就产生一个幅度固定的电脉冲（动作电位）；达不到阈值则不产生。动作电位沿轴突单向传播，幅度不衰减。
- ◆ (4)传递信号。突触电脉冲到达轴突末梢时，释放神经递质，将信号传递给下一个神经元或效应器（如肌肉）。



3.1.1 生物神经元

5. 启示

- ◆ (1)信息整合：人工神经元采用“加权求和 + 偏置”来模拟信号的空间总和。
- ◆ (2)阈值激活：人工神经元使用激活函数（如Sigmoid等）来模拟动作电位的“全或无”特性。
- ◆ (3)连接权重：突触强度对应人工神经网络中的权重，且均可通过学习调整。
- ◆ (4)多层结构：多个神经元通过突触连接形成网络，对应多层人工神经网络。

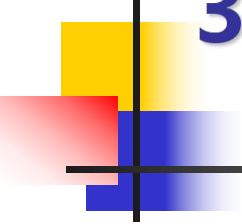


3.1.2 人工神经元

1. 含义

MP神经元 (McCulloch-Pitts Neuron)

- ◆ 1943年由沃伦·麦卡洛克 (Warren McCulloch) 和沃尔特·皮茨 (Walter Pitts) 受到生物神经元的启发，但进行了极大简化（省去了时间动态、神经调质、树突非线性计算、脉冲编码等步骤）。
- ◆ 提出的第一个人工神经元的数学模型，奠定了人工神经网络的理论基础。
- ◆ 由于MP神经元权重和阈值固定、不直接支持实数值输入等局限，在现代神经网络中很少使用。

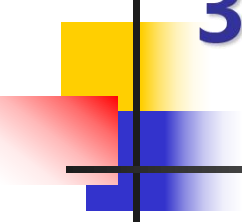


3.1.2 人工神经元

1. 含义

人工神经元

- ◆ 是人工神经网络中的基本计算单元，是对生物神经元功能的数学抽象。
- ◆ 接收多个输入信号（通常带有可调节的权重），对输入加权求和后加上偏置项，再通过一个激活函数（如阶跃函数、Sigmoid、ReLU等）产生输出。



3.1.2 人工神经元

2. 特点

- ◆ (1)参数可学习。
- ◆ (2)非线性映射能力。
- ◆ (3)结构简单，计算高效。
- ◆ (4)可组合性。
- ◆ (5)分布式表示。
- ◆ (6)统一数学模型。

3.1.2 人工神经元

3. 结构

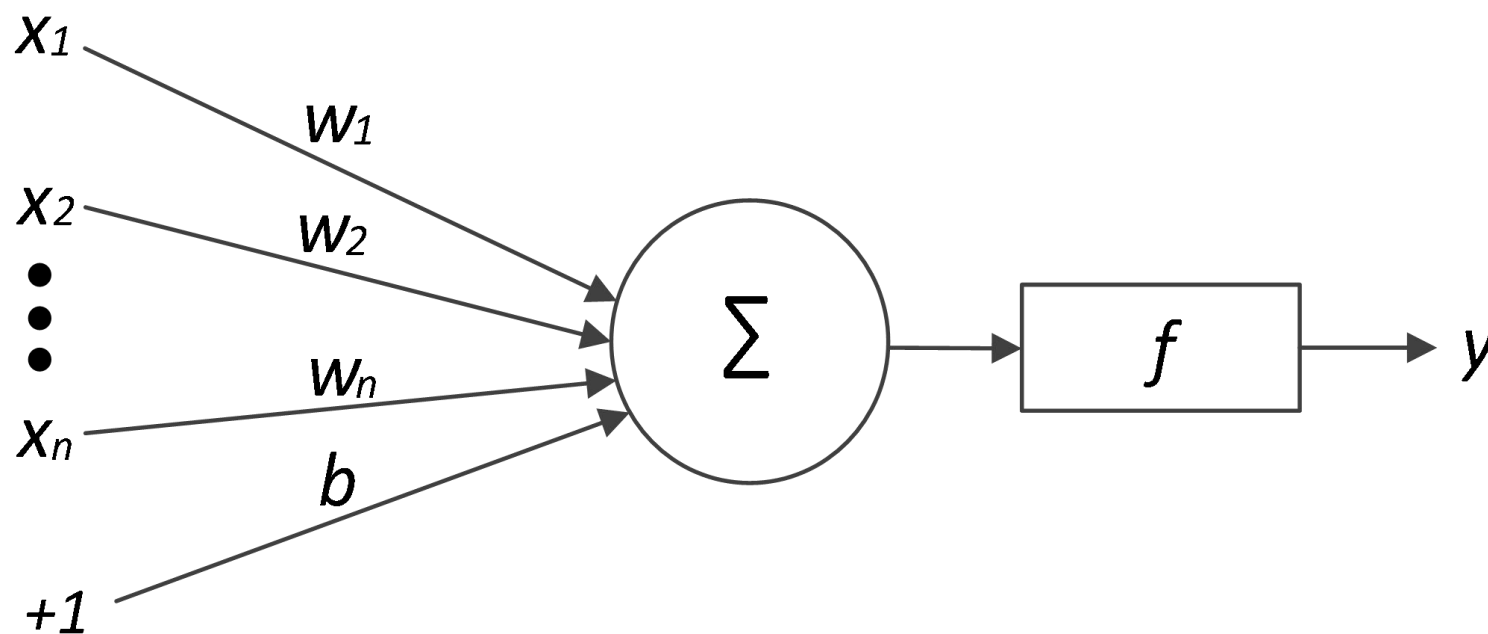


图 人工神经元

3.1.2 人工神经元

3. 结构

- ◆ (1)输入 $x_1, x_2, \dots, x_n$: 接收来自前一层或其他来源的数值信号。
- ◆ (2)权重 $w_1, w_2, \dots, w_n$: 每个输入对应的可学习参数, 决定输入的重要性, 模拟突触连接强度, 由人工设计。
- ◆ (3)偏值 b : 可学习的常数, 使神经元能够调整激活阈值。
- ◆ (4)求和器 Σ : 计算加权 $z = \sum_{i=1}^n w_i x_i + b$
- ◆ (5)激活函数 f : 对 z 施加非线性变换, 得到最终输出 $y = f(z)$

3.1.2 人工神经元

4. 数学模型

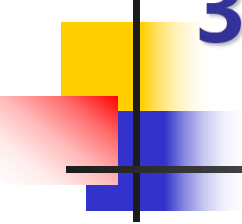
人工神经元运算分为两步：

◆ (1)线性组合。计算公式如下：

$$z = W^T X + b = \sum_{i=1}^n w_i x_i + b$$

◆ (2)非线性激活。计算公式如下：

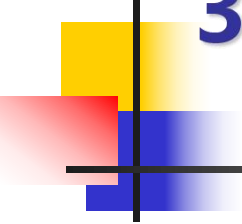
$$y = f(z)$$



3.1.2 人工神经元

5. 局限性

- ◆ (1)单个神经元表达能力有限。一个神经元只能学习一个线性决策面，必须依靠多层网络才能突破此限制。
- ◆ (2)需要大量标注数据。与生物神经元相比，人工神经网络的学习效率极低，通常需要成千上万的样本来泛化。
- ◆ (3)缺乏时间动态。标准人工神经元输出仅依赖当前输入，无内部状态或记忆。
- ◆ (4)忽略生物神经元的复杂特性。不包含树突非线性计算、脉冲编码、神经调质、突触时间可塑性等机制，这些生物特性可能在效率上更优。
- ◆ (5)解释性差。单个神经元的权重和激活值通常难以直接解释为有意义的语义特征，网络整体像一个“黑箱”。



3.1.3 激活函数

1. 作用

- ◆ (1)引入非线性。通过非线性变换，使网络能学习到非线性的复杂模式。
- ◆ (2)特征选择。激活函数可以决定神经元的激活状态，帮助神经网络筛选和选择重要的特征。
- ◆ (3)控制输出范围。一些激活函数能够限制输出范围，如Sigmoid()函数将输出限制在(0, 1)之间。
- ◆ (4)缓解梯度问题。合适的激活函数能缓解梯度消失或爆炸问题，确保深层网络可训练。
- ◆ (5)增强模型的鲁棒性。非线性激活函数（如ReLU）可以抑制噪声的传播，例如过滤掉负值输入。

3.1.3 激活函数

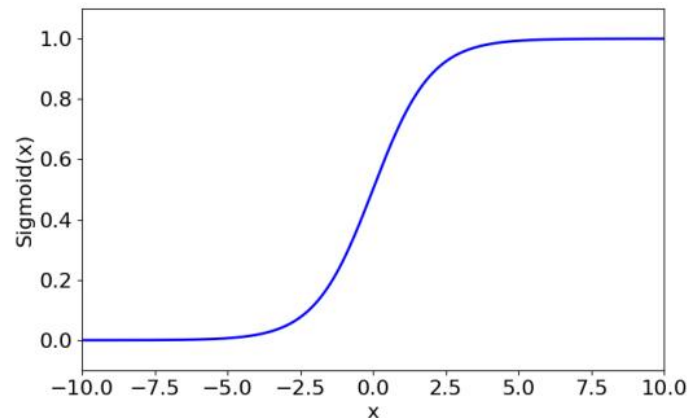
2. 常见激活函数

(1) Sigmoid函数

- ◆ 将一个实数映射到[0,1]区间，常用于二分类问题。

$$\text{Sigmoid}(x) = \frac{1}{1+e^{-x}}$$

- ◆ Sigmoid函数的图形如图所示。其形状如S形，也称为S形生长曲线。
- ◆ 优点：平滑，容易求导；具有良好的解释性。
- ◆ 缺点：反向传播涉及除法，计算量大，容易出现梯度消失的情况，限制了深层网络的训练。



3.1.3 激活函数

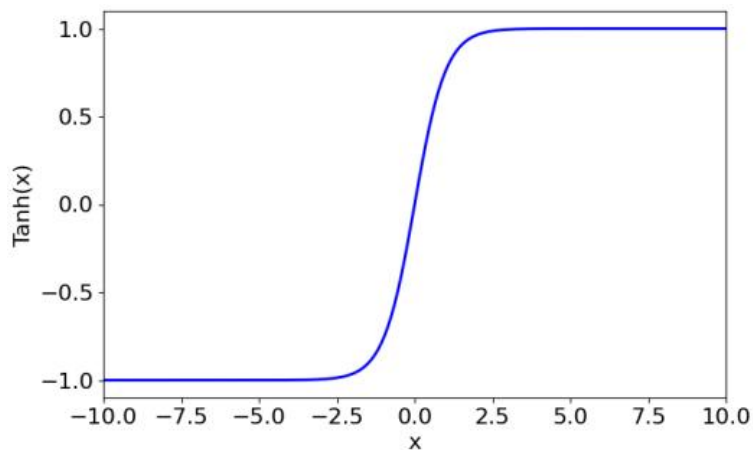
2. 常见激活函数

(2) Tanh函数

- ◆ Tanh函数是Sigmoid()函数的改进，结果区间位于 $[-1,1]$ 之间。

$$\text{Tanh}(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- ◆ Tanh函数的图形如图所示。Tanh函数在区间 $(-1,1)$ 内是单调递增的奇函数，图像关于原点对称。
- ◆ 优点：收敛速度快，不容易出loss值晃动，在分类任务中使用较多。
- ◆ 缺点：无法解决容易出现梯度消失的情况，计算相对复杂。



3.1.3 激活函数

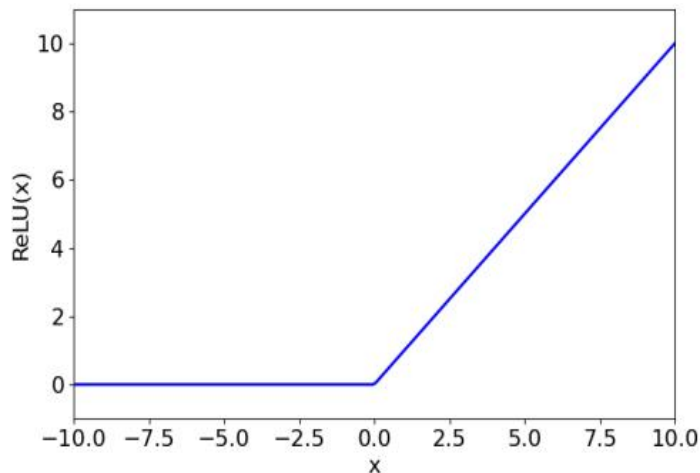
2. 常见激活函数

(3) ReLU函数

- ◆ ReLU函数是Sigmoid函数和Tanh函数的替换函数，是深度学习领域最重要的突破技术之一。

$$Relu(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases}$$

- ◆ ReLU函数的图形如图所示。
- ◆ 优点：不存在梯度消失问题，计算速度低，收敛速度快。
- ◆ 缺点：非零均值输出；容易出现神经元死亡和负信息丢失的情况。



3.1.3 激活函数

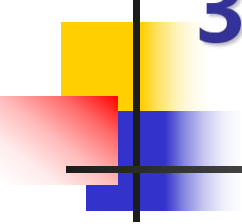
2. 常见激活函数

(4) Softmax函数

- ◆ Softmax函数将多个神经元的输出映射到 $(0,1)$ 区间内且和为1，可以看成概率来理解。

$$\text{Softmax}(x_i) = \frac{e^i}{\sum_j e^j}$$

- ◆ 优点：将任意实数向量转换为概率分布（和为1，非负），具有概率解释，适用于多分类。
- ◆ 缺点：对异常值敏感（指数放大），输出受输入尺度影响，当类别数多时计算开销大。

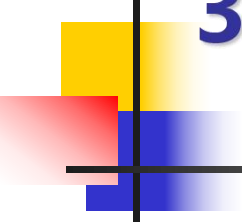


3.1.3 激活函数

3. 激活函数选择

(1) 隐藏层

- ◆ 默认首选 ReLU。计算简单、正区间梯度恒定，有效缓解梯度消失，适合大多数深度网络。若出现大量神经元“死亡”，可换用 Leaky ReLU 或 ELU。
- ◆ 避免使用 Sigmoid / Tanh 作为隐藏层激活。它们在饱和区梯度接近零，容易导致梯度消失，且输出非零均值（Sigmoid）会拖慢收敛。仅在 RNN 门控等特定结构中保留使用。
- ◆ 对极深网络或 Transformer 架构。可考虑 Swish / GELU，它们平滑、非单调，往往能带来更好性能，但计算开销略高。



3.1.3 激活函数

3. 激活函数选择

(2)输出层

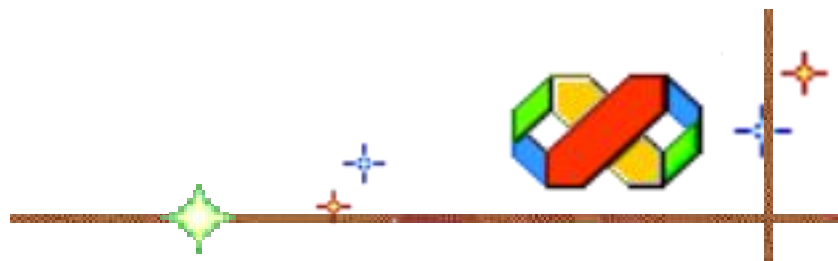
- ◆ 回归任务：通常线性激活函数或者不使用激活函数。
- ◆ 二分类任务：通常选择Sigmoid函数，输出概率值。
- ◆ 多分类任务：通常选择Softmax函数，输出各类别概率且和为1。

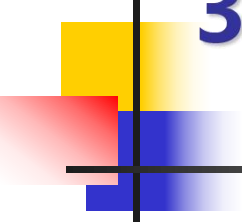
3.2 单层感知机和多层感知机

3.2.1 单层感知机

3.2.2 多层感知机

3.2.3 损失函数

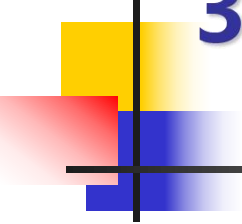




3.2.1 单层感知机

1. 定义

- ◆ 单层感知机是最简单的人工神经网络模型，由弗兰克·罗森布拉特1958年提出。
- ◆ 单层感知机由输入层直接连接输出层（没有隐藏层），使用阶跃激活函数进行二分类，且只能解决线性可分问题，本质上是一个线性二分类器。
- ◆ 单层感知机是历史上第一个具有学习能力的神经网络模型，奠定了现代神经网络的基础。



3.2.1 单层感知机

2. 特点

- ◆ (1)线性决策边界：只能学习线性可分的模式（如AND、OR函数），无法解决XOR（异或）这类非线性问题。
- ◆ (2)学习算法：采用感知机学习规则（迭代更新权重），通过梯度下降的思想修正错误分类样本的权重。
- ◆ (3)收敛性：若训练数据线性可分，算法在有限步内收敛；否则权重会震荡，无法收敛。

3.2.1 单层感知机

3. 模型结构

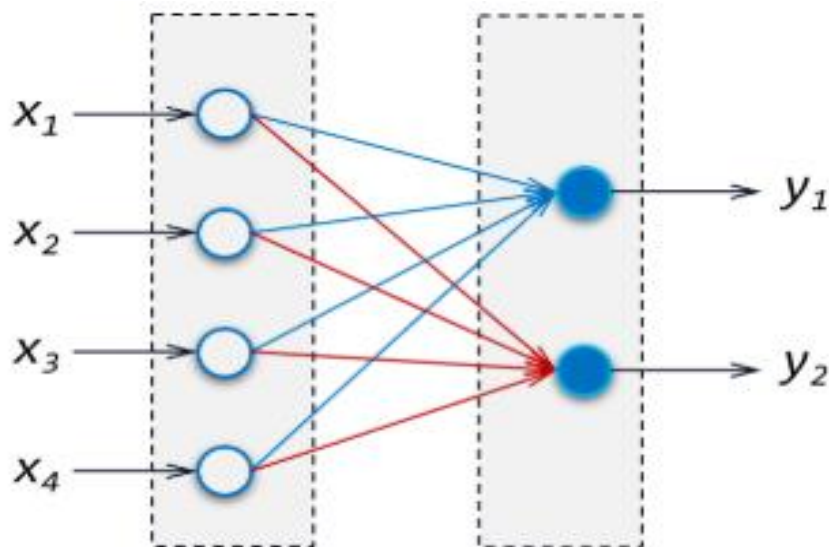


图 单层感知机

3.2.1 单层感知机

3. 模型结构

构成：

- ◆ 输入层：接收原始特征向量 $(x_1, x_2, \dots, x_n)$ ，每个输入节点对应一个特征。这一层仅负责传递数据，不进行计算。
- ◆ 输出层：包含一个或多个神经元（通常用于二分类时为一个神经元）。每个输入节点与输出神经元之间都有连接，每条连接对应一个权重 w_i ，此外还有一个独立的偏置项 b_i （或视为与常数连接的权重）。
- ◆ 所有输出神经元共享同一层输入，但彼此之间没有连接。

3.2.1 单层感知机

4. 数学公式

假定：

- ◆ 输出层：包含 m 个神经元；
- ◆ 输入向量： $X = [x_1, x_2, \dots, x_n]^T$
- ◆ 权重矩阵： W ，其中元素 w_{ji} 为第 j 个神经元与第 i 个输入之间的连接权重
- ◆ 偏置向量：
- ◆ 第 j 个神经元的偏置向量 $b = [b_1, b_2, \dots, b_m]^T$ 与法如下：
$$y_j = f(\sum_{i=1}^n w_{ji}x_i + b_j) = f(W_j^T + b_j)$$
- ◆ 其中， W_j^T 是权重矩阵的第 j 行。

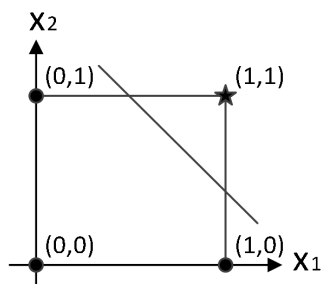
3.2.1 单层感知机

4. 数学公式

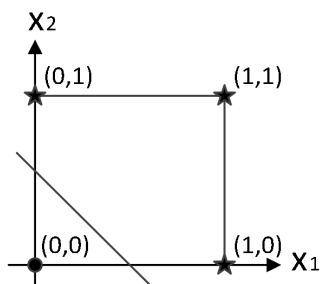
◆ 【例】使用单层感知机实现逻辑运算。

表 逻辑运算规则

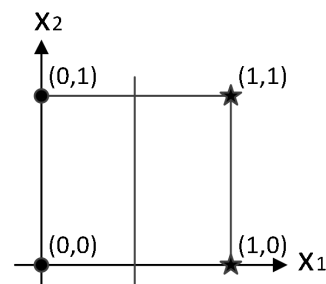
x_1	x_2	$x_1 \wedge x_2$	$x_1 \vee x_2$	$\tilde{x}_1$
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0



“与”运算



“或”运算



“非”运算

图 求解逻辑运算

3.2.1 单层感知机

4. 数学公式

- ◆ 由图可知，对于上述三种逻辑运算，总可以找到一条直线将上述三种逻辑运算输出为1和输出为0的结果分开。
 - ◆ (1)逻辑 $\wedge$ （与）运算。 $w_1=1, w_2=1, b=-1.5$ ，对应直线为 $x_1+x_2-1.5=0$ 。
 - ◆ (2)逻辑 $\vee$ （或）运算。 $w_1=1, w_2=1, b=-0.5$ ，对应直线为 $x_1+x_2-0.5=0$ 。
 - ◆ (3)逻辑 $\sim$ （非）运算。 $w_1=1, w_2=0, b=-1$ ，对应直线为 $x_1-1=0$ 。
- ◆ 直线将二维平面分为两部分，上部或右部为激发区（输出为1），下部或左部为抑制区（输出为0）。这是一个线性可分问题，能够通过单层感知机实现。

3.2.1 单层感知机

4. 数学公式

程序代码：

- ◆ `from sklearn.linear_model import Perceptron`
- ◆ `import numpy as np`
- ◆ `# 生成训练数据`
- ◆ `X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])`
- ◆ `y = np.array([0, 0, 0, 1])`
- ◆ `# 创建单层感知机模型`
- ◆ `clf = Perceptron(max_iter=1000, tol=1e-3)`
- ◆ `clf.fit(X, y) # 训练模型`
- ◆ `print("权重:", clf.coef_)`
- ◆ `print("截距:", clf.intercept_)`
- ◆ `print("预测:", clf.predict(X))`

运行结果：

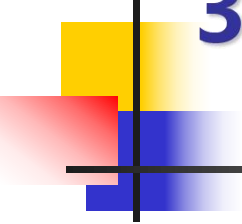
- ◆ 权重: `[[2. 2.]]`
- ◆ 截距: `[-2.]`
- ◆ 预测: `[0 0 0 1]`区（输出为0）。

3.2.1 单层感知机

5. 学习算法

在线更新方式

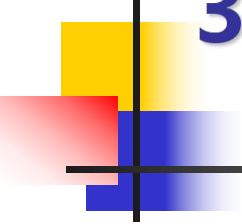
- ◆ (1)初始化。设置权重和偏置 b 为较小的随机数或零，选择学习率 $\eta \in (0,1]$ （通常取 0.1 或 1）。
- ◆ (2)迭代训练（最多 E 轮或直到收敛）。对每个训练样本 (x,y) :
 - ◆ A.计算当前输出 $\hat{y}$;
 - ◆ B.如果 $\hat{y} \neq y$ （分类错误），更新权重和偏置：
$$w \leftarrow w + \eta \cdot (y - \hat{y}) \cdot x$$
$$b \leftarrow b + \eta \cdot (y - \hat{y})$$
 - ◆ C.如果分类正确，则 w 和 b 保持不变。
- ◆ (3)终止条件：所有样本均被正确分类，或达到最大迭代次数停止。



3.2.1 单层感知机

6. 局限性

- ◆ 无法解决非线性问题：这是最核心的局限，最著名的是不能解决异或（XOR）问题。
- ◆ 输出结果单一：只能输出确定的类别（如+1或-1），无法输出属于某一类的概率值。
- ◆ 对异常值敏感：训练过程受错误分类样本影响大，少量异常值可能严重干扰分类超平面学习。
- ◆ 多分类扩展困难：处理多分类需要需要训练多个感知机，不如现代模型直接支持多分类。



3.2.2 多层感知机

1. 定义

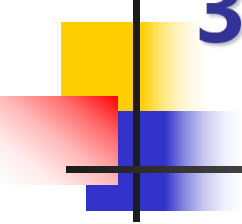
- ◆ 多层感知机 (Multilayer Perceptron, MLP) 是一种前馈人工神经网络, 是人工神经网络中最基础、最重要的结构之一。MLP通过引入隐藏层和非线性激活函数, 突破了单层感知机的线性局限, 能够拟合任意复杂的非线性函数, 并借助反向传播算法实现了高效训练。
- ◆ MLP的核心思想是通过隐藏层的非线性激活函数 (如 ReLU、Sigmoid 等) 将输入空间逐层映射到更复杂的特征空间, 使得在输出层可以用一个线性分类器 (或回归器) 解决原本非线性不可分的问题。
- ◆ ML是现代深度学习的基石, 虽然近年来卷积神经网络 (CNN) 和 Transformer 在特定领域表现更优, 但MLP依然因其简单通用而广泛应用于各类机器学习任务中。

3.2.2 多层感知机

1. 定义

形象化比喻 - 工厂流水线

- ◆ (1)原料入库（输入层）。原始数据进入。
- ◆ (2)多车间加工（隐藏层）。每个车间：工人按配方（权重）将生成的半成品送入下一个车间 → 质检门（激活函数）只让合格品进入下一环节。
。
- ◆ (3)成品包装（输出层）。产出最终结果（分类/预测）。
- ◆ (4)质检反馈（损失函数）。对比成品与标准，计算误差。
- ◆ (5)倒查改进（反向传播+优化）。从后向前追溯：哪个工人配方不对？调整配方（梯度下降），下次做得更好。
- ◆ (6)重复迭代（训练）。反复生产→反馈→调参，产品越做越精。



3.2.2 多层感知机

2. 特点

- ◆ 解决非线性问题：这是最关键的突破。凭借隐藏层和非线性激活函数（如ReLU、Sigmoid），MLP可以拟合任意复杂的非线性决策边界，MLP可以轻松解决。
- ◆ 强大的学习能力：MLP是通用近似定理的一个实例。理论上，一个足够宽的MLP（即隐藏层有足够多神经元）可以以任意精度逼近任何复杂的连续函数。
- ◆ 自动特征提取：隐藏层能够自动学习并组合输入数据中的底层特征，形成更高层次的抽象表示，减少了对人工特征工程的依赖。
- ◆ 多输出能力：输出层可以包含多个神经元，天然支持多分类任务（如使用Softmax函数）。

3.2.2 多层感知机

3. 模型结构

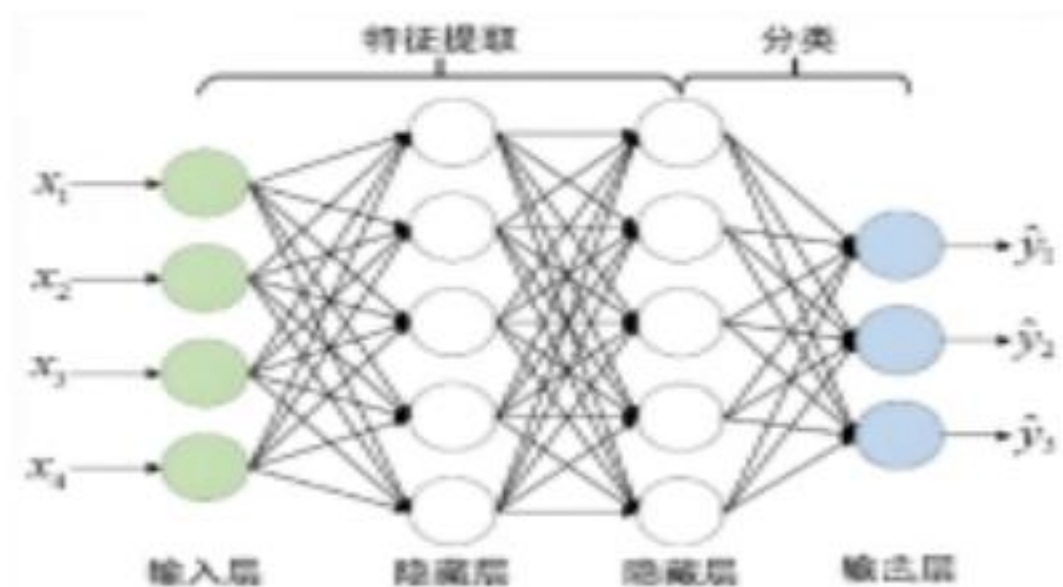
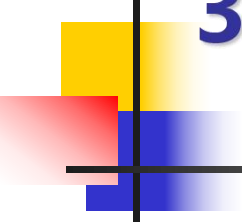


图 多层感知机



3.2.2 多层感知机

3. 模型结构

- ◆ MLP是一种前馈神经网络，由以下三层（或更多）构成，层与层之间全连接（每个神经元与下一层的所有神经元相连），每一层都有自己的权重矩阵和偏置向量。（如图所示）：
- ◆ 输入层：接收原始特征，不进行计算。
- ◆ 隐藏层：一个或多个，每个神经元对输入进行加权求和后，通过非线性激活函数产生输出。
- ◆ 输出层：根据任务（分类、回归）给出最终预测。

3.2.2 多层感知机

4. 工作流程

(1)前向传播（计算预测值）

- ◆ 输入层：接收原始特征向量 X 。
- ◆ 隐藏层：逐层计算
- ◆ 线性变换：计算加权和： $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$
- ◆ 函数激活：使用激活函数 $f(l)$ 对线性变换的结果 $z(l)$ 进行激活： $a^{(l)} = f_{(l)}(z^{(l)})$
- ◆ 其中， $f(l)$ 为第 l 层的激活函数， $a^{(0)} = x$
- ◆ 输出层：最后一层产生预测输出 y （如分类概率或回归值）。

(2)计算损失

- ◆ 用损失函数 $L(y, \hat{y})$ 衡量预测值 $\hat{y}$ 与真实标签 y 的差距（如交叉熵、均方误差）。

3.2.2 多层感知机

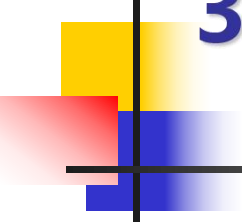
4. 工作流程

(3)反向传播（计算梯度）

- ◆ 从输出层逆向逐层计算损失对各层参数（权重W、偏置b）的偏导数（梯度），利用链式法则传递误差。

(4)参数更新

- ◆ 使用梯度下降（或其他优化器）更新参数，其中是 η 学习率。
- ◆ 权重参数更新： $W^{(l)} = W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}}$
- ◆ 偏置参数更新： $b^{(l)} = b^{(l)} - \eta \frac{\partial L}{\partial b^{(l)}}$
- ◆ 重复步骤 1-4 多个 epoch，直到损失收敛或达到停止条件。



3.2.2 多层感知机

5. 案例

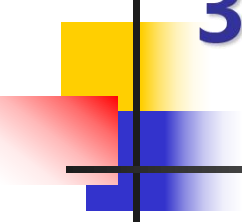
- ◆ 【例】使用sklearn实现MLP求解逻辑异或运算。逻辑异或运算的规则为：当输入层的2个输入不同，结果为1；否则为0。
- ◆ (1)分析：
- ◆ 多层感知机可以解决 XOR 问题，其过程如下：
 - ◆ 隐藏层将输入非线性变换到新空间，使数据线性可分；
 - ◆ 输出层在新空间完成线性分类；
 - ◆ 最终网络能完美拟合 XOR 真值表。

3.2.2 多层感知机

5. 案例

(2)网络结构

- ◆ 采用一个简单的两层网络（输入层、隐藏层、输出层）：
- ◆ 输入层：2 个神经元 (x_1, x_2)
- ◆ 隐藏层：2 个神经元，使用 Sigmoid 激活函数
- ◆ 输出层：1 个神经元，使用 Sigmoid 激活函数
- ◆ 损失函数：使用均方误差 (MSE) : $L = (1/2) * (y_{\text{pred}} - y_{\text{true}})^2$

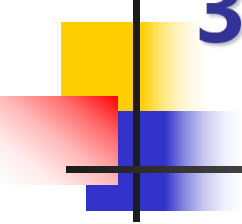


3.2.2 多层感知机

5. 案例

程序代码:

- ◆ `import numpy as np`
- ◆ `from sklearn.neural_network import MLPClassifier`
- ◆ `# 1. 定义XOR数据集`
- ◆ `# 输入: (x1, x2) 的四种组合`
- ◆ `X = np.array([[0, 0],`
- ◆ `[0, 1],`
- ◆ `[1, 0],`
- ◆ `[1, 1]])`
- ◆ `# 输出: XOR的真值表`
- ◆ `y = np.array([0, 1, 1, 0])`

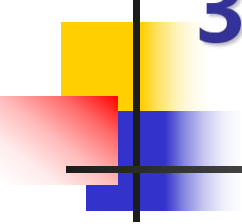


3.2.2 多层感知机

5. 案例

程序代码:

```
◆ # 2. 创建MLP模型
◆ model = MLPClassifier(
◆     hidden_layer_sizes=(8,),           # 1个隐藏层, 8个神经元
◆     activation='relu',                 # 使用ReLU激活函数
◆     solver='adam',                     # 自适应矩估计优化器
◆     max_iter=1000,                     # 最大迭代次数
◆     random_state=42                    # 固定随机种子, 确保可复现
◆ )
```



3.2.2 多层感知机

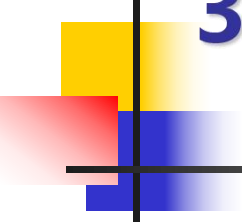
5. 案例

程序代码:

- ◆ # 3. 训练模型
- ◆ `model.fit(X, y)`
- ◆ # 4. 预测并评估
- ◆ `print("模型准确率:", model.score(X, y))`
- ◆ `print("预测结果:", model.predict(X))`
- ◆ `print("期望结果:", y)`

运行结果:

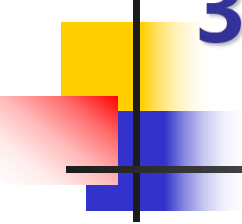
- ◆ 模型准确率: 1.0
- ◆ 预测结果: [0 1 1 0]
- ◆ 期望结果: [0 1 1 0]



3.2.2 多层感知机

6. 局限性

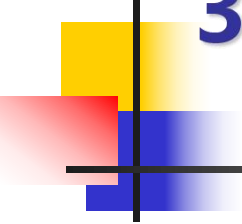
- ◆ (1)参数爆炸：全连接结构导致参数量巨大，尤其在高维输入（如图像）时易过拟合且训练缓慢。
- ◆ (2)忽略数据结构：无法利用图像的空间局部性、时序依赖等先验知识，需人工特征工程。
- ◆ (3)训练困难：深层MLP易出现梯度消失或梯度爆炸，难以优化。
- ◆ (4)对输入敏感：特征尺度不一致或分布变化时性能下降，需精细归一化。
- ◆ (5)泛化瓶颈：在复杂高维任务（如图像、语音、文本）上效果远不如CNN/RNN/Transformer。



3.2.2 多层感知机

7. 应用场景

- ◆ (1)分类任务：如垃圾邮件识别、手写数字识别（MNIST）、图像分类、信用评级。
- ◆ (2)回归任务：如房价预测、股票价格预测、销量预测。
- ◆ (3)模式识别：如人脸识别、语音识别（早期）、文本分类（情感分析、主题分类）。
- ◆ (4)工业与金融：欺诈检测、风险评估、客户细分、质量控制。
- ◆ (5)其他领域：医学诊断（基于结构化数据）、推荐系统（辅助特征学习）。



3.2.3 损失函数

1. 作用

- ◆ (1)量化模型预测误差。损失函数通过数学公式将抽象的“模型好坏”转化为可优化的数值指标。
- ◆ (2)指导模型参数优化。通过最小化损失函数，模型逐步调整权重，使预测更接近真实值。
- ◆ (3)应对数据噪声与过拟合。在损失函数中加入L1/L2正则化项，惩罚过大参数，防止过拟合。
- ◆ (4)评估模型性能。通过比较不同模型的损失值，可以选择性能最佳的模型。

3.2.3 损失函数

2. 常见损失函数

(1)均方误差 (MSE - Mean Squared Error, MSE)

- ◆ MSE是指参数估计值与参数真值之差平方的期望值，用来评估模型的好坏程度，MSE越小说明预测模型精确度越高。MSE是最常用、最经典的回归损失函数。平方根形式是均方根误差（RMSE）。
- ◆ 公式：
$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$
- ◆ 其中， y_i 是第*i*个真实值， $\hat{y}_i$ 是第*i*个预测值， n 是样本数。
- ◆ 应用场景：数据中异常值不多的大多数回归任务。
- ◆ 优点：直观，计算简单。
- ◆ 缺点：对异常值过于敏感，可能导致模型对噪声数据过拟合。

3.2.3 损失函数

2. 常见损失函数

(2) 平均绝对误差 (Mean Absolute Error, MAE)

- ◆ MAE 衡量的是预测值和真实值之间的绝对误差的平均值。
- ◆ 数学公式: $MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$
- ◆ 其中, y_i 是第 i 个真实值, $\hat{y}_i$ 是第 i 个预测值, n 是样本数。
- ◆ 应用场景: 在对异常值不敏感的回归任务中使用。
- ◆ 优点: 对异常值不敏感, 能够更加稳定地反映模型性能。
- ◆ 缺点: 在优化过程中, 绝对值函数不可导, 求解困难。

3.2.3 损失函数

2. 常见损失函数

(3) Huber损失 (HL)

◆ MSE和MAE的折衷。

◆ 数学公式：

$$L_{\delta} = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & |y - \hat{y}| \leq \delta \\ \delta(|y - \hat{y}| - \frac{1}{2}\delta) & \text{其它} \end{cases}$$

◆ 其中， y 为真实值， $\hat{y}$ 为预测值， δ 为阈值超参数（通常需要调节）。

◆ 应用场景：具有异常值的回归任务。需要同时兼顾MSE的优化效率和MAE的鲁棒性。

◆ 优点：在误差较小时，它表现为MSE；在误差较大时，它表现为MAE。

◆ 缺点：需要选择一个合适的 δ 值。

3.2.3 损失函数

2. 常见损失函数

(4)交叉熵损失

- ◆ 分类问题中最核心、最常用的损失函数族，它衡量两个概率分布之间的差异。
- ◆ A.二分类交叉熵损失 (Cross-Entropy Loss, BCE)
- ◆ 数学公式: $BCE = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$
- ◆ 应用场景: 广泛用于分类问题, 尤其是在处理二分类和多分类问题时。
- ◆ 优点: 当预测值与实际值差距较大时, 损失增加显著, 有助于快速学习。
- ◆ 缺点: 在某些情况下可能会遇到数值稳定性问题 (例如, 对数函数内的参数接近0) 。

3.2.3 损失函数

2. 常见损失函数

(4)交叉熵损失

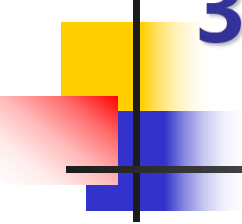
- ◆ B.多分类交叉熵损失 (Categorical Cross-Entropy Loss, CCE)
- ◆ 数学公式: $CCE = -\sum_{i=1}^k y_i \log(\hat{y}_i)$
- ◆ 其中, 真实标签 $y = [y_1, y_2, \dots, y_k]$ 为one-hot编码, 模型输出概率分布 $\hat{y} = [\hat{y}_1, \hat{y}_2, \dots, \hat{y}_k]$
- ◆ 应用场景: 多分类问题, 特别是类别互斥的情况。
- ◆ 优点: 直接针对多分类问题, 优化目标明确。
- ◆ 缺点: 在类别不平衡的数据集中可能表现不佳。

3.2.3 损失函数

2. 常见损失函数

(5) Kullback-Leibler 散度 (KL Divergence, KL)

- ◆ KL散度衡量两个概率分布之间的差异。
- ◆ 数学公式: $KL = \sum_i p(i) \log \frac{P(i)}{Q(i)}$
- ◆ 应用场景: 多用于模型压缩、变分自编码器 (VAE) 等需要比较两个分布的场景。
- ◆ 优点: 对概率分布之间的微小差异非常敏感。
- ◆ 缺点: 对稀疏分布的概率模型不稳定。

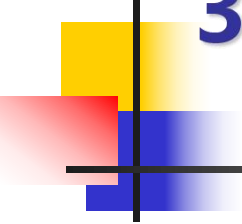


3.2.3 损失函数

2. 损失函数选择

(1) 回归任务

- ◆ 均方误差 (MSE): 最常用。对较大误差惩罚更重, 模型会努力避免大偏差; 缺点是对异常值敏感。
- ◆ 平均绝对误差 (MAE): 对异常值更鲁棒, 但梯度恒定, 大误差时收敛可能稍慢。
- ◆ Huber Loss: 融合两者优点。小误差时用MSE (平滑), 大误差时用MAE (抗异常)。
- ◆ 对于特别复杂的任务, 通常需要自定义损失。



3.2.3 损失函数

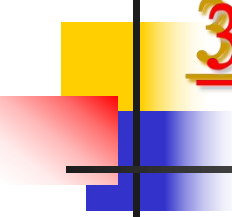
2. 损失函数选择

(2) 分类任务

- ◆ 二分类任务：通常选择二元交叉熵 (Binary Cross-Entropy)，同时使用配合 Sigmoid 输出。
- ◆ 多分类任务：通常选择分类交叉熵 (Categorical Cross-Entropy)，同时使用 Softmax 输出。
- ◆ 多标签分类：通常选择二元交叉熵配合 Sigmoid 对每个类别独立，然后将损失求和或平均。

(3) 复杂任务

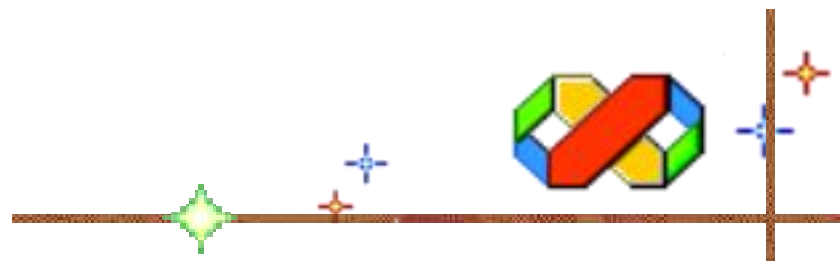
- ◆ 对于特别复杂的任务，通常需要自定义损失。

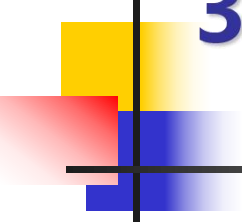


3.3 模型评估和优化

3.3.1 优化器

3.3.2 参数初始化

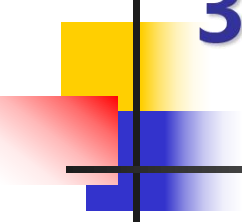




3.3.1 优化器

1. 优化器的作用

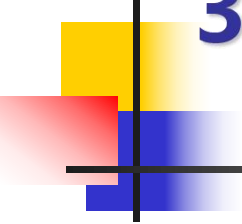
- ◆ (1)参数更新机制。优化器定义了模型参数（如权重和偏置）如何根据梯度进行更新。
- ◆ (2)加速训练收敛。通过引入动量（如SGD + Momentum）、自适应学习率（如Adam、RMSProp）等机制，优化器能显著加快模型收敛速度，避免在平坦区域停滞。
- ◆ (3)提升训练稳定性。防止梯度震荡或爆炸，帮助跳出局部最优和鞍点，通过动量累积平滑更新路径
- ◆ (4)自适应调节学习率。不同参数可能需要不同的更新步长。Adam、AdaGrad等优化器能为每个参数动态调整学习率，对频繁更新的参数减小步长，稀疏特征则加大步长，提升整体效率。
- ◆ (5)支持大规模模型训练。现代优化器（如AdamW）专为大模型设计，能稳定处理数十亿参数的更新，在BERT、GPT等LLM训练中广泛应用。



3.3.1 优化器

1. 工作原理

- ◆ 所有优化器都建立在梯度下降的思想之上。
- ◆ 假设要最小化损失函数是 $L(\theta)$ ，其中 θ 是模型参数。
- ◆ 标准 SGD（随机梯度下降）的工作原理：每次随机选一个样本（或一小批样本）计算梯度，然后更新参数。
- ◆ (1)计算梯度（反向传播）。通过求导计算损失函数 $L(\theta)$ 对参数 θ 的梯度 $\Delta L(\theta)$ 。
- ◆ (2)更新参数。按照素雅方向调整参数 $\theta_{\text{new}} = \theta_{\text{old}} - \eta \Delta L(\theta_{\text{old}})$ ，其中 η 是学习率。

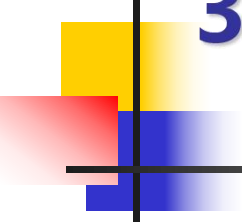


3.3.1 优化器

3. 优化器类型

(1) 基本梯度下降类（含动量变体）

- ◆ 直接使用梯度（或梯度的累积方向）更新参数，学习率全局固定或手动衰减。这类优化器的收敛路径相对稳定，泛化能力较强，但对学习率敏感，调参较复杂。
- ◆ 代表优化器如下：
 - ◆ SGD（随机梯度下降）：最基础的实现，每次沿负梯度方向更新。
 - ◆ Momentum：加入动量项，累积历史梯度以加速收敛并平滑震荡。
 - ◆ NAG（Nesterov 加速梯度）：在动量基础上，利用“预估位置”的梯度进行更前瞻的更新。

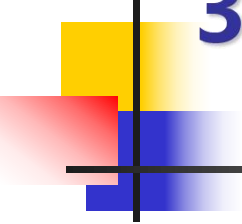


3.3.1 优化器

3. 优化器类型

(2) 自适应学习率类

- ◆ 为每个参数单独维护一个学习率，根据梯度历史自动调整更新步长。这类优化器能够减少手动调参负担，但对某些任务可能出现学习率不稳定或泛化性能下降。
- ◆ 代表优化器如下：
 - ◆ AdaGrad：累积梯度平方和，频繁更新的参数学习率衰减快，适合稀疏数据。
 - ◆ RMSProp：用指数移动平均替代累积平方和，解决 AdaGrad 学习率过早消失的问题，适合非平稳目标（如 RNN）。



3.3.1 优化器

3. 优化器类型

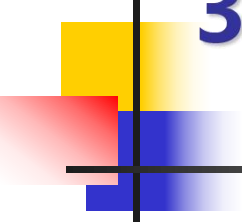
(3) 自适应矩估计类 (Adam 及其变体)

- ◆ 同时利用梯度的一阶矩（动量）和二阶矩（自适应学习率），并加入偏差校正。这类优化器结合了两者的优点，是目前深度学习的默认选择，超参数相对鲁棒。
- ◆ 代表优化器如下：
 - ◆ Adam：最通用的优化器，收敛快、稳定性好。
 - ◆ AdamW：修正 Adam 中 L2 正则化与自适应学习率的耦合，解耦权重衰减，泛化性更强（Transformer 常用）。
 - ◆ Nadam：Adam + Nesterov 动量，收敛速度略有提升。
 - ◆ AMSGrad：对二阶矩取最大值，防止学习率意外增大。

3.3.1 优化器

4. 优化器选择

- ◆ (1)第一次尝试 / 快速原型：推荐Adam (或 AdamW)，特点是收敛快、超参数鲁棒。
- ◆ (2)图像分类 CNN (ResNet、VGG 等)：推荐SGD + Momentum，其泛化性能通常优于 Adam，但需要精细调整学习率和衰减策略。
- ◆ (3)Transformer 模型 (BERT、GPT、ViT)：推荐AdamW，其解耦权重衰减，正则化更规范。
- ◆ (4)RNN / LSTM：推荐RMSProp 或 Adam，其应对非平稳梯度，避免梯度爆炸/消失。
- ◆ (5)稀疏数据 (NLP 词嵌入、推荐系统)：推荐AdaGrad 或 Adam，自动为稀有特征增大更新步长。
- ◆ (6)在线学习 / 非平稳目标：推荐RMSProp，其学习率自适应且不单调衰减。



3.3.1 优化器

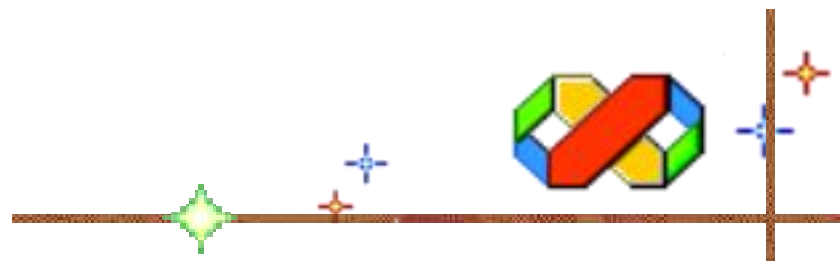
5. 学习率选择

- ◆ (1)SGD + Momentum: 推荐0.01 或 0.1。
- ◆ (2)Adam: 推荐 $1e-3$ (0.001)。
- ◆ (3)AdamW: 推荐 $1e-3$ 或 $3e-4$ (Transformer)。
- ◆ (4)RMSProp: 推荐 $1e-3$ 或 $1e-2$ 。
- ◆ (5)AdaGrad: 推荐0.01。

3.4 拟合问题和正则化

3.4.1 拟合问题

3.4.2 正则化



3.4.1 拟合问题

1. 什么是拟合

- ◆ 拟合 (Fitting) 是机器学习、统计学中的核心概念，就是从数据中找规律，让数学模型去适应（逼近）观测数据的过程。
- ◆ 给定训练集 $\{(x_1, y_1), (x_1, y_1), \dots, (x_n, y_n)\}$ 拟合就是寻找一个函数 $f(x; \theta)$ ，使得预测值 $\hat{y}_i = f(x_i; \theta)$ 与真实值 y_i 之间的误差尽可能小，即：

$$\min_{\theta} \sum_{i=1}^n L(f(x_i; \theta), y_i)$$

- ◆ 其中， L 是损失函数，如平方误差、交叉熵损失等。

3.4.1 拟合

2. 拟合类型

- ◆ 按照模型和数据的拟合效果，拟合可分为三种：欠拟合 (Underfitting)、适当拟合 (Good Fitting)和过拟合 (Overfitting)。

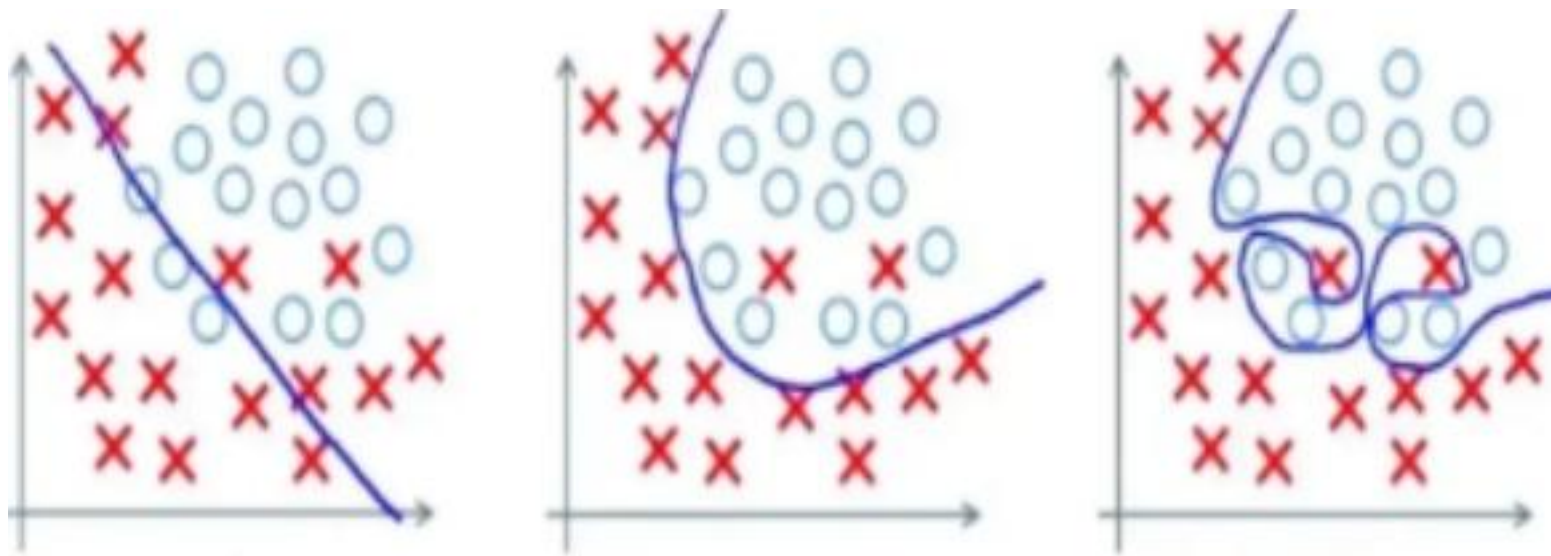
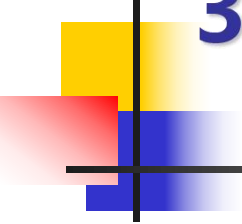


图 拟合类型（欠拟合，适当拟合，过拟合）

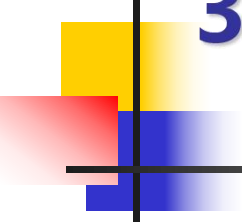


3.4.1 拟合

2. 拟合类型

(1) 欠拟合

- ◆ 欠拟合是指模型在训练集上的误差很大，在测试集（新数据）的预测效果差，其本质是模型过于简单，表达能力不足，未能捕捉到数据中的真实规律（如图中的a）。
- ◆ 欠拟合就像一个学生习题做的太少，平时测试（训练集）和期末考试（测试集）成绩都不理想。
- ◆ 常见原因。包括：模型复杂度过低（如层数太少、神经元过少）；特征工程不足（输入特征未能包含有效信息）；正则化过强（如过大的权重衰减或 dropout 概率）；训练不充分（如学习率过低、迭代轮数太少）。
- ◆ 解决方法。包括：增加模型复杂度（加深/加宽网络）；减少正则化强度；提取更有意义的特征；延长训练时间或调整优化器。



3.4.1 拟合

2. 拟合类型

(2)适当拟合

- ◆ 适当拟合是指模型的训练误差和测试误差都较低且接近，其本质是模型学到的数据的真实规律（如图中的b）。
- ◆ 适当拟合就像一个学生平时做了一定量的习题，而且掌握了其中的知识和做题技巧，因而平时测试（训练集）成绩很好，期末考试（测试集）成绩也很好。

3.4.1 拟合

2. 拟合类型

(3)过拟合 (Overfitting)

- ◆ 过拟合是指模型在训练集上表现极好，但在测试集（新数据）上误差很大，其本质是模型在训练数据上学习到了“特定样本的噪声和偶然规律”，而非数据的真实规律，因而泛化能力弱（如图中的c）。
- ◆ 过拟合就像一个学生平时把习题及答案都背下来了，平时测试（训练集）成绩很好，期末考试（测试集）成绩则很不理想。
- ◆ 常见原因。包括：模型复杂度过高（如层数过多、参数过多等）；训练数据不足或代表性差；训练时间过长（迭代轮数过多）；数据存在噪声或异常值。
- ◆ 解决方法。包括：增加数据量或进行数据增强；降低模型复杂度（如减少层数、参数和特征等）；添加正则化（如L1/L2正则化、Dropout、早停等）；集成学习（如Bagging、随机森林等）。

3.4.1 正则化

1. 核心思想

- ◆ 正则化的核心思想是在损失函数中加入惩罚项，限制模型复杂度。
- ◆ 原始损失函数（经验风险）：

$$L(\theta) = \frac{1}{N} \sum_{i=1}^n \text{loss}(y_i, \hat{y}_i)$$

- ◆ 正则化后的损失（结构风险）：

$$L_{reg}(\theta) = L(\theta) + \lambda R(\theta)$$

3.4.1 正则化

2. 正则化方法

L1 正则化 (Lasso)

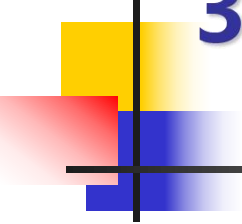
- ◆ L1 正则化的目标是使权重稀疏（很多权重变为0），自动进行特征选择，适用于高维稀疏特征场景（如文本分类）。计算公式如下：

$$R(\theta) = \sum_i |\omega_i|$$

(2)L2 正则化 (Ridge / 权重衰减)

- ◆ L2 正则化的目标是使权重整体变小但非零，抑制极端权重值，提高模型平滑性，是大多数神经网络默认的正则化方法。计算公式如下：

$$R(\theta) = \sum_i \omega_i^2$$



3.4.1 正则化

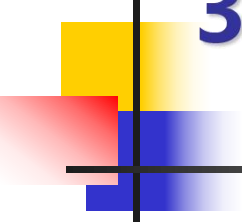
1. 核心思想

(3) 丢弃率 (Dropout)

- ◆ 训练时随机“丢弃”一部分神经元（将其输出置0），相当于每次迭代训练一个不同的子网络，迫使神经元不过度依赖特定特征，增强泛化能力。
- ◆ 深度神经网络最有效的正则化手段之一。
- ◆ 隐藏层Dropout通常取值 0.5，输入层通常取值 0.2。

(4) 早停 (Early Stopping)

- ◆ 在训练过程中监控验证集损失，当验证损失连续多个 epoch 不再下降甚至上升时，停止训练并回滚到最佳模型。
- ◆ 这种方法相当于限制了有效训练轮数（一种隐式正则化）。



3.4.1 正则化

1. 核心思想

(5)数据增强 (Data Augmentation)

- ◆ 通过对训练数据施加合理的变换（如图像旋转、裁剪、翻转等），生成更多多样化的训练样本，从而扩充有效数据集大小，降低过拟合。

(6)批归一化 (Batch Normalization)

- ◆ 主要作用是加速收敛和稳定训练，但因其引入了噪声（小批量统计量的随机性），也具有一定的正则化效果，有时可减少 Dropout 的依赖。